

Reading Python Error Messages (CodeGrade)

- Due Mar 26, 2025 by 11:59pm
- Points 1
- Submitting an external tool
- Available until Mar 26, 2025 at 11:59pm

This assignment was locked Mar 26, 2025 at 11:59pm.

 <https://github.com/learn-co-curriculum/python-p3-reading-python-error-messages>  <https://github.com/learn-co-curriculum/python-p3-reading-python-error-messages/issues/new>

Learning Goals

- Read the different parts of an error message.
- Identify common types of errors.

Key Vocab

- **Interpreter:** a program that executes other programs. Python programs require the Python interpreter to be installed on your computer so that they can be run.
- **Python Shell:** an interactive interpreter that can be accessed from the command line.
- **Data Type:** a specific kind of data. The Python interpreter uses these types to determine which actions can be performed on different data items.
- **Exception:** a type of error that can be predicted and handled without causing a program to crash.
- **Code Block:** a collection of code that is interpreted together. Python groups code blocks by indentation level.
- **Function:** a named code block that performs a sequence of actions when it is called.
- **Scope:** the area in your program where a specific variable can be called.

Introduction

In this lab, you'll be reading error messages from tests. This lab is designed so that both running the files *and* running the test suite via the `pytest` command will show the error messages for you to decode. Moving forward though, you'll be reading error messages mainly through running the test suite.

Reading Error Messages

Let's start by running some of the Python code in the `lib/` folder to produce an error message. Run this in your terminal:

```
$ python lib/a_name_error.py
```

Error messages have 3 parts:

```
File "lib/a_name_error.py",  
line 3, in <module>  
    print(hello_world)  
NameError: name 'hello_world' is not defined
```

The location of the error, the "where":

```
"lib/a_name_error.py", line 3, in <module>:
```

- `"lib/a_name_error.py"` is the file the error occurred in.
- `line 3` is the line of code with the error.
- `<module>` is the scope of the error.

The type of error, the "who":

```
NameError:
```

This is a [Python Error Type](https://docs.python.org/3/tutorial/errors.html)  [_\(https://docs.python.org/3/tutorial/errors.html\)_](https://docs.python.org/3/tutorial/errors.html).

The description, the "why":

```
name 'hello_world' is not defined
```

The interpreter does the best job it can to tell you what it thinks went wrong.

You've solved games of *Clue* with less information. This is one of the best parts of programming: debugging and fixing errors. It's like you're a detective solving a crime. The only bad thing is that more often than not, you're also the criminal that caused the error in the first place.

Errors are clues, and reading them is the interpreter telling you what to do to fix the program and move on.

Three Common Error Types

Syntax Errors

Syntax errors are pretty self-explanatory: they're the result of incorrect syntax. Thankfully, they're usually followed by a guess about the location of the error. For instance:

```
2 * #
```

Will result in:

```
File "<stdin>", line 1
  2 * #
    ^
```

```
SyntaxError: invalid syntax
```

Here, Python is saying that on line 1, there is a missing number (every `*` operator must be preceded and followed by a number or variable with a numerical value). Always read the full details of syntax errors and look for line numbers, which usually appear at the beginning of the error message.

Logic Errors

Logic errors are often difficult to find because they are not perceived as errors by the Python interpreter itself. To find a logic error, a programmer will often need to comb through their code line by line. Debugging tools such as `pdb` (which we will cover later on in Phase 3) are very helpful for locating and fixing logic errors.

```
count = 1
while count < 100:
    print("%i" % count)
```

Will produce the following output:

```
1
1
1
1
1
1
1
1
...
```

The programmer here forgot to increase the count during each iteration of the `while` loop. This is perfectly valid Python code, so the interpreter will not throw an error, but the loop will continue forever until it is manually stopped by the user. (The easiest way to do this in the terminal is `ctrl + c`)

Exceptions

Exceptions  (<https://docs.python.org/3/library/exceptions.html>) cover a wide variety of errors that you may see when running Python code. Our `NameError` from earlier is one example of a Python exception.

Exceptions pop up when the interpreter knows what to do with a piece of code but is unable to complete the action. A key difference between the other types of errors and exceptions is that the Python interpreter can continue reading your application after an exception- you just need to tell it what to expect.

There are many types of exceptions in Python; here are a few of the most common:

AssertionError

An `assert()` statement tells the interpreter that the code inside of it must proceed without error or exception. If an assertion fails, an `AssertionError` is raised.

```
assert(1 == 2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
```

IndexError and KeyError

`IndexErrors` arise when you try to access an element at an index past the end of a list. Key errors relate to `dict` objects in Python (similar to JSON objects). If a key is referenced but does not exist, this exception is thrown.

```
> list = [0, 1, 2, 3, 4]
> dict = {'a':1, 'b':2, 'c':3}

> list[10]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range

> dict['d']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'd'
```

NameError

A `NameError` arises when a name is referenced before it has been defined.

```
> flatiron_school
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'flatiron_school' is not defined
```

TypeError

TypeError arise when an operation or function is applied to an object of the wrong type.

```
> wrong_type = 'abc' + 123
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Instructions

To get started, run `pytest -x` to run the first test in the test suite. Use the error messages to guide your work:

- Read the errors. Scroll through the entire output to get a sense of what the failures are trying to tell you. What does the error mean? How can we fix it?
- Each error prints out a **stack trace**, which points to where the code failed and attempts to follow it *up the stack* — that is, through the bits of code that ran leading up to the failure. You can use these stack traces to pinpoint which line(s) of code need your attention.
- These stack traces can also point you to which files you should run to get a better sense of the errors.

Fix the errors in each of the files in `lib/`. Then confirm the fix by running `pytest` again.

Commit and push your work using `git` when all of your tests have passed!